Laboratory Guide

Lesson 05 — BLE Server on Arduino

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva

ESTGA, University of Aveiro

Estimated duration: 2 hours

Contents

Context **2**

Learning Outcomes **2**

1 Assembling the Peripheral **3**

2 Command-Line Verification **3**

3 Notifications and Visualisation **5**

Questions for Discussion **6**

References **6**

Context

Lesson 04 established the GAP and GATT concepts and traversed, using command-line tools, the service and characteristic hierarchy of a peripheral. This session builds the peripheral: an Arduino fitted with the HM-11 module exposes sensor readings through BLE notifications, and a client collects them and forwards them for display.

The two programs required are provided in the `codigo/` folder and are meant to be loaded and run, not written. The student's work consists of the assembly, the command-line verification and the construction of the visualisation flow in Node-RED.

Material

- Arduino Uno with HM-11 BLE shield and DHT11 sensor.
- The `DHTlib` Arduino library, the same one installed in Lesson 02. The provided sketch will not compile without it.
- Raspberry Pi with BlueZ and the `bluepy` library installed.
- Node-RED on the Raspberry Pi, with the `@flowfuse/node-red-dashboard` package.

All equipment is that already used in previous lessons.

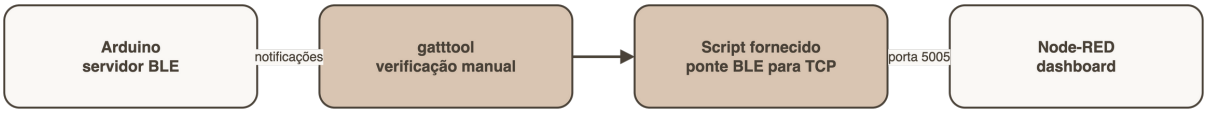


Figure 1: *Architecture of the complete chain*

Learning Outcomes

- Assemble a BLE peripheral and confirm that it advertises correctly;
- Confirm by command line, before running any program, that the expected services and characteristics are exposed;
- Distinguish, in practice, reading by polling from reception by notification;
- Integrate the readings received into a display panel built without writing code.

1. Assembling the Peripheral

25 min

Objective

Mount the HM-11 shield and the sensor on the Arduino, and confirm operation through the serial monitor before involving BLE communication.

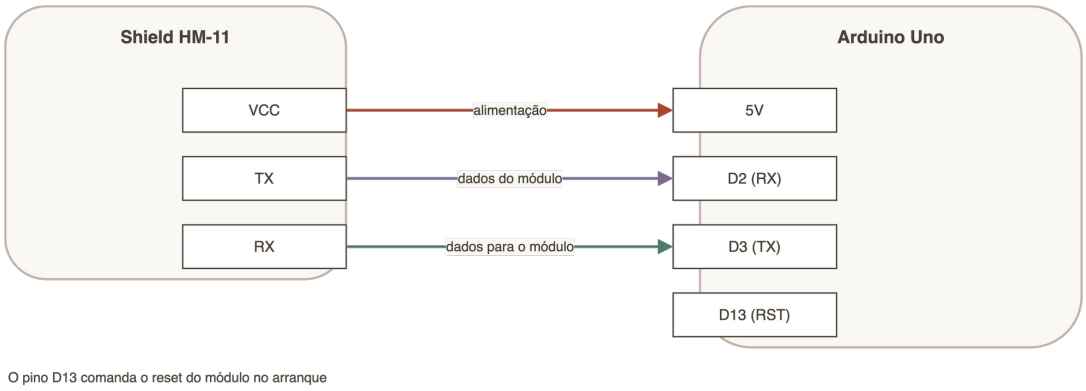


Figure 2: HM-11 shield connections to the Arduino

Module	Arduino	Function
VCC	5V	Power supply
TX	D2	Data from the module to the Arduino
RX	D3	Data from the Arduino to the module
RST	D13	Reset at start-up
GND	GND	Ground
DHT11 DATA	D7	Temperature and humidity reading

1. Make the connections with the Arduino disconnected from the USB cable.
2. Load the file `codigo/servidor_ble.ino`, provided with this guide.
3. Open the serial monitor at 9600 bit/s and send the character `X`. A line with temperature and humidity separated by a comma should be returned.

Why verify through the serial monitor first

At this stage BLE communication is not yet involved. If the sensor reading does not work here, any failure observed later would be wrongly attributed to the BLE link. This order of verification is the working method recommended in the lecture.

2. Command-Line Verification

30 min

Objective

Confirm, from the Raspberry Pi and without resorting to any program, that the peripheral advertises and exposes the expected characteristics.

```
hcitool dev
sudo hcitool lescan
sudo gatttool -b <ADDRESS> -I
```

In `gatttool` interactive mode:

```
connect
primary
characteristics
char-write-req <handle> 58
```

1. Record the address of the HM-11 module obtained from the scan.
2. Record the UUID of the service exposed by the module and the *handle* of the data characteristic.
3. The `char-write-req` command sends the value `0x58`, which corresponds to the character `X`. Confirm that the Arduino responds.

Reading the result

The values returned appear in hexadecimal. Conversion to text is immediate: each pair of digits corresponds to one character of the response sent by the Arduino.

3. Notifications and Visualisation

45 min

Objective

Run the provided bridge, which enables BLE notifications and makes them available on a local TCP port, and build in Node-RED the panel that displays them.

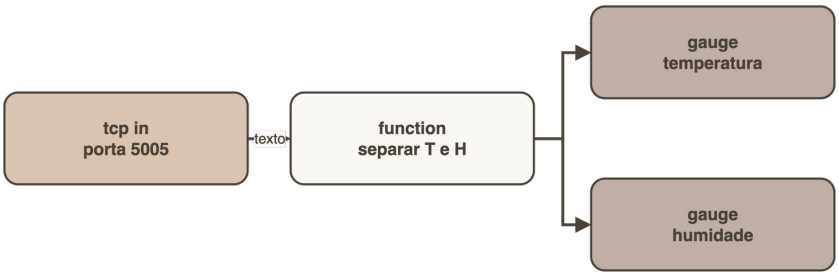


***Figure 3:** Node-RED flow to be built*

Running the bridge

```
python3 codigo/ponte_ble_tcp.py <ADDRESS>
```

The program connects to the peripheral, enables notifications and listens on `127.0.0.1:5005`. The lines received are also shown in the console, which allows operation to be confirmed before involving Node-RED.

Building the flow

1. Drag a `tcp in` node. Configure *Connect to* mode, host `127.0.0.1`, port `5005` and delivery of strings delimited by `\n`.
2. Drag a `function` node with two outputs and paste the provided content:

```
var v = msg.payload.split(",");
return [ { payload: parseFloat(v[0]) },
         { payload: parseFloat(v[1]) } ];
```

3. Connect the first output to a `ui-gauge` labelled *Temperature* and the second to a `ui-gauge` labelled *Relative humidity*.
4. Press `Deploy` and open `http://127.0.0.1:1880/dashboard`.

Where the energy saving lies

The Arduino only transmits when it receives a request, and the bridge receives the responses as notifications, without repeatedly polling the characteristic. Comparing this behaviour with that of periodic reading allows the difference discussed in the lecture to be observed.

Further work

Change the request interval in the provided bridge from one to ten seconds and observe the effect on the panel. What trade-off arises between data freshness and radio traffic?

Questions for Discussion

1. Why was a software serial port required on the Arduino?
2. What is the difference between notification and indication, and which is being used?
3. Which GATT role does the Arduino play and which does the Raspberry Pi play?
4. If the Arduino stopped responding, after how long would the connection be considered lost, and which parameter determines that value?

References

- Gonçalves, P. *Apresent5 — BLE Dev* and *guia 4 — arduinoBLE*, TCIoT, previous editions.
- bluepy library documentation: `https://ianharvey.github.io/bluepy-doc/`
- Node-RED TCP nodes: `https://nodered.org/docs/user-guide/`